Goal

The goal of this exercise is to find the maximum number in an array. This should be done using 3 different methods. 1. Using a nested for loop on the CPU. 2. Using atomic operations on the GPU. 3. using reduction technique on the GPU. The second and third step will utilize the high thread count of the GPU for parallel operations. In this exercise, an array of size 2048 is used. The last goal is to compare performance of each of these methods to each other.

Finding the maximum

To implement method number 1 a for loop is used on the CPU to iterate over the entire array. Each element is compared to the previous maximum element, if the current element is higher it is saved as the new maximum. Note that by using a for loop, the process happens sequential. To explain the second method, we first have to explain what atomic operations are. With traditional parallel operations, access to common memory from multiple parallel threads can cause race conditions. Using atomic operations a thread claims a common resource (memory) until the calculation is complete, then after the resource is declaimed the other threads use it. As long as the resource is claimed, no other thread can utilize it. Using the atomic max operation each element is compared to the first element, the highest element is saved in the first element of the array. Each thread on the GPU will compare Handle one element of the array, thus 2048 threads are needed in this exercise. This operation is partly parallel as each thread handles one element, but the claiming and declaiming of memory makes this a sequential operation. The last method is using the reduction technique. Figure: 1 indicates how the reduction technique is implemented. Reduction allows for operations to happen in parallel by using different elements in the array at one instance to find the maximum. The first row of Figure: 1 shows that 7 different operations can happen in parallel, the second operation allows for 3 parallel operations. Between each step the threads need to be synchronized, for example thread 0 in step 2 depends on thread 1 being finished. Before continuing to the next step in the reduction, each thread has to finish the previous step. By using reduction, the operation is run partly in parallel and there needs to be no claiming of memory. To ensure the operations can work properly, we need to use the steps discussed in the previous lab to set up the memory and create the kernel functions.

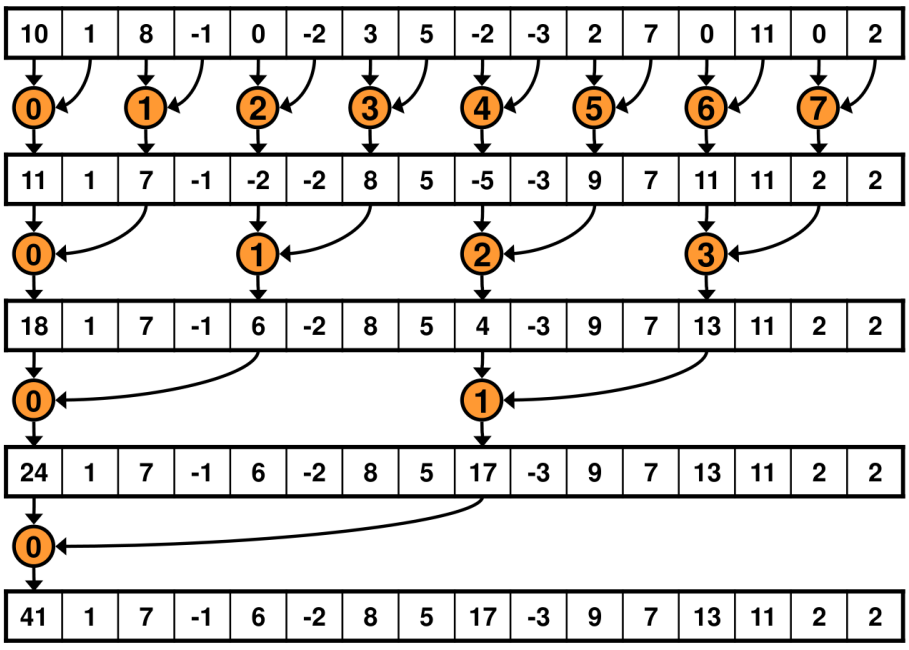


Figure 1: Reduction method

Speed of finding the maximum

When comparing the speed, we find that the reduction method is 4 x faster than the atomic add process. This could be since the operation runs in parallel compared to the atomic method that runs sequential. The CPU method is 4x slower than the reduction method, we would expect the reduction method to be faster due to the parallel nature of the method. The CPU is in this case faster, which could be explained as the unoptimized reduction code or the speed of the CPU outweighing the GPU speed.



Conclusion

In This test we see that for an array size of 2048 that the CPU is faster than the GPU using atomic method or reduction.

